

Schema Validation in Mongoose

In real-world applications, we do not allow users to enter any type of data.

We use **Validation** to ensure that only valid data is stored in the database.

Mongoose provides built-in validation through Schema options.

Basic Schema

```
const mongoose = require("mongoose");

const studentSchema = new mongoose.Schema({
  name: String,
  age: Number,
  course: String
});

const Student = mongoose.model("Student", studentSchema);
```

Q1. How do we make a field mandatory?

Validation: required

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  age: Number
});
```

Valid

```
await Student.create({
  name: "Shahid",
```

```
    age: 21
  });
```

Invalid

```
await Student.create({
  age: 21
});
```

Error:

Path `name` is required.

Q2. How do we set a default value?

Validation: default

```
const studentSchema = new mongoose.Schema({
  name: String,
  course: {
    type: String,
    default: "MERN"
  }
});
```

Example

```
await Student.create({
  name: "Shahid"
});
```

Stored document:

```
{
  "name": "Shahid",
  "course": "MERN"
}
```

Q3. How do we set a minimum value?

Validation: min

```
const studentSchema = new mongoose.Schema({
  age: {
    type: Number,
    min: 18
  }
});
```

Invalid

```
await Student.create({
  age: 15
});
```

Error:

Path `age` (15) is less than minimum allowed value (18).

Q4. How do we set a maximum value?

Validation: max

```
const studentSchema = new mongoose.Schema({
  age: {
    type: Number,
    max: 60
  }
});
```

Invalid

```
await Student.create({
  age: 70
});
```

Q5. How do we set a minimum length for a string?

Validation: minlength

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    minlength: 3
  }
});
```

Invalid

```
await Student.create({
  name: "A1"
});
```

Q6. How do we set a maximum length for a string?

Validation: maxlength

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    maxlength: 20
  }
});
```

Invalid

```
await Student.create({
  name: "ThisNameIsMoreThanTwentyCharacters"
});
```

Q7. How do we allow only specific values?

Validation: enum

```
const studentSchema = new mongoose.Schema({
  course: {
    type: String,
    enum: ["MERN", "Java", "Python"]
  }
});
```

Valid

```
await Student.create({
  course: "MERN"
});
```

Invalid

```
await Student.create({
  course: "C++"
});
```

Error:

```
`C++` is not a valid enum value.
```

Q8. How do we remove extra spaces automatically?

Validation: trim

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    trim: true
  }
});
```

```
}  
});
```

Input:

```
{  
  name: "  Shahid  "  
}
```

Stored:

```
{  
  name: "Shahid"  
}
```

Q9. How do we convert text into lowercase automatically?

Validation: lowercase

```
const studentSchema = new mongoose.Schema({  
  email: {  
    type: String,  
    lowercase: true  
  }  
});
```

Input:

```
{  
  email: "SHAHID@GMAIL.COM"  
}
```

Stored:

```
{  
  email: "shahid@gmail.com"  
}
```

Q10. How do we convert text into uppercase automatically?

Validation: uppercase

```
const studentSchema = new mongoose.Schema({  
  course: {  
    type: String,  
    uppercase: true  
  }  
});
```

Input:

```
{  
  course: "mern"  
}
```

Stored:

```
{  
  course: "MERN"  
}
```

Q11. How do we validate email format?

Validation: match

```
const studentSchema = new mongoose.Schema({  
  email: {  
    type: String,  
    match: /^[^\s@]+@[^\s@]+\.[^\s@]+$/  
  }  
});
```

```
    }  
  });
```

Valid

```
shahid@gmail.com
```

Invalid

```
shahidgmail.com
```

Q12. How do we create our own custom validation?

Validation: validate

```
const studentSchema = new mongoose.Schema({  
  age: {  
    type: Number,  
    validate: {  
      validator: function(value) {  
        return value >= 18;  
      },  
      message: "Age must be 18 or above"  
    }  
  }  
});
```

Invalid

```
await Student.create({  
  age: 15  
});
```

Error:

```
Age must be 18 or above
```

Complete Professional Schema

```
const studentSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true,
    trim: true,
    minlength: 3,
    maxlength: 30
  },

  age: {
    type: Number,
    required: true,
    min: 18,
    max: 60
  },

  email: {
    type: String,
    required: true,
    lowercase: true,
    match: /^[^\\s@]+@[^\\s@]+\\. [^\\s@]+$/
  },

  course: {
    type: String,
    enum: ["MERN", "Java", "Python"],
    default: "MERN"
  }
});
```

Quick Revision

Required Field

```
required: true
```

Default Value

```
default: "MERN"
```

Number Validation

```
min: 18  
max: 60
```

String Length Validation

```
minlength: 3  
maxlength: 20
```

Allowed Values

```
enum: ["MERN", "Java"]
```

Remove Spaces

```
trim: true
```

Convert Text

```
lowercase: true  
uppercase: true
```

Pattern Matching

```
match: /regex/
```

Custom Validation

```
validate: {}
```

Interview Definition

Schema Validation is a feature of Mongoose that ensures only valid and properly formatted data is stored in MongoDB by applying rules such as required, min, max, enum, match, and custom validations.